SEPM – Software Engineering & Project Management

CSE 'E' – 8-Mark Student-Friendly Answers

Based on: *Software Engineering, 9th Edition* by Ian Sommerville

Easy language • Point-wise format • Exam-ready

Q1. Explain the role of system boundaries and explain context model for MHC-PMS with neat diagram.

Ref: Sommerville, SE 9th Ed., Chapter 5, Section 5.1 and Figure 5.1

Introduction

System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system. Context models are used to illustrate the operational context of a system — they show what lies outside the system boundaries.

Definition of System Boundaries

System boundaries define what is **inside** the system (features to be developed) and what is **outside** (part of the external environment). Establishing clear boundaries is one of the first and most critical steps in requirements engineering because it directly affects the scope, cost, and timeline of the project.

Role and Importance of System Boundaries

1. **Scope Definition:** System boundaries help engineers and stakeholders clearly understand what features and functionalities will be built and what will be excluded from development. This prevents scope creep.
2. **Decision Making:** Helps engineers decide which features to build internally and which to delegate or interface with external systems.
3. **Social and Political Factors:** Boundaries are not purely technical decisions. They may be influenced by organizational politics, departmental budgets, management preferences, or the desire to keep development work within a specific team.
4. **Data Management Decisions:** Boundaries help decide whether the system should maintain its own database (providing faster access and independence) or rely on external databases (avoiding data duplication but creating dependencies).
5. **Cost and Resource Allocation:** Clear boundaries help in accurate estimation of development effort, time, and resources required.
6. **Interface Identification:** Boundaries help identify which external systems need to be interfaced with, enabling early planning of communication protocols and APIs.

What is a Context Model?

A context model is a high-level architectural view that shows the system as a single entity surrounded by other systems and external entities with which it interacts. It does NOT show the internal structure of the system — only its relationships with the external environment.

The Context Model for MHC-PMS

MHC-PMS (Mental Health Care Patient Management System) is a specialized information system used in clinics that treat patients with mental health problems.

- **Central System:** MHC-PMS — the system being developed, shown at the center.
- **Connected External Systems:**
  - **Admissions System:** Manages patient admissions and hospital bed allocation. MHC-PMS queries this system when a patient needs to be admitted.
  - **Prescription System:** Handles the generation and management of medication prescriptions. MHC-PMS sends prescription requests to this system.
  - **Patient Record System:** A general healthcare database that stores comprehensive patient health records. MHC-PMS retrieves and updates patient information here.
  - **Statistics System:** Used for generating management reports, research analysis, and statistical data for health authorities.
  - **Appointments System:** Manages scheduling of patient consultations with doctors and therapists.
  - **Staff Management System:** Contains information about medical staff, their schedules, and availability.

Key Points About the Diagram

- The arrows show the direction of data flow between MHC-PMS and external systems.
- The diagram uses UML notation with the system represented as a rectangle.
- External systems are shown as separate entities outside the main system boundary.

Advantages of Context Models

1. Provides a clear overview of system interactions at a glance.
2. Helps stakeholders understand what data flows in and out.
3. Useful for identifying integration requirements early.
4. Serves as a communication tool between technical and non-technical stakeholders.

The Unified Modeling Language

The Unified Modeling Language is a set of 13 different diagram types that may be used to model software systems. It emerged from work in the 1990s on object-oriented modeling where similar object-oriented notations were integrated to create the UML. A major revision (UML 2) was finalized in 2004. The UML is universally accepted as the standard approach for developing models of software systems. Variants have been proposed for more general system modeling.

http://www.SoftwareEngineering-9.com/Web/UML/

In the third case, where models are used as part of a model-based development process, the system models have to be both complete and correct. The reason for this is that they are used as a basis for generating the source code of the system. Therefore, you have to be very careful not to confuse similar symbols, such as stick and block arrowheads, that have different meanings.

5.1 Context models

At an early stage in the specification of a system, you should decide on the system boundaries. This involves working with system stakeholders to decide what functionality should be included in the system and what is provided by the system's environment. You may decide that automated support for some business processes should be implemented but others should be manual processes or supported by different systems. You should look at possible overlaps in functionality with existing systems and decide where new functionality should be implemented. These decisions should be made early in the process to limit the system costs and the time needed for understanding the system requirements and design.

In some cases, the boundary between a system and its environment is relatively clear. For example, where an automated system is replacing an existing manual or computerized system, the environment of the new system is usually the same as the existing system's environment. In other cases, there is more flexibility, and you decide what constitutes the boundary between the system and its environment during the requirements engineering process.

For example, say you are developing the specification for the patient information system for mental healthcare. This system is intended to manage information about patients attending mental health clinics and the treatments that have been prescribed. In developing the specification for this system, you have to decide whether the system should focus exclusively on collecting information about consultations (using other systems to collect personal information about patients) or whether it should also collect personal patient information. The advantage of relying on other systems for patient information is that you avoid duplicating data. The major disadvantage, however, is that using other systems may make it slower to access information. If these systems are unavailable, then the MHC-PMS cannot be used.

Figure 1: Context Model for MHC-PMS

Q2. Process model for involuntary detention.

Ref: Sommerville, SE 9th Ed., Chapter 5, Section 5.1 and Figure 5.2

Introduction

Process models (also called interaction models or activity models) are used to show how a system interacts with its environment during specific business processes. They reveal the sequence of activities and the flow of control during operations.

Definition

A process model shows the **sequence of activities** involved in completing a real-world task. For MHC-PMS, it specifically tracks the legal and medical process of detaining a mental health patient against their will when they pose a danger to themselves or others.

Context: Involuntary Detention

Involuntary detention is a serious legal process in mental health care where:

- A patient is held in a secure facility without their consent
- This is only allowed when the patient is deemed dangerous
- Strict legal procedures must be followed to protect patient rights
- Multiple stakeholders must be informed and involved

UML Activity Diagram Components

The process uses a UML Activity Diagram to show the flow of control:

1. **Start Node:** A filled black circle indicating where the process begins.
2. **Activities (Actions):** Rounded rectangles representing tasks such as:
   - "Confirm detention decision"
   - "Inform patient of legal rights"
   - "Inform next of kin"
   - "Find secure bed"
   - "Transfer patient"
3. **Decision Points (Guards):** Diamond shapes with conditions in brackets like *[Dangerous]* or *[Not dangerous]* that determine the flow path.
4. **Fork/Join Bars:** Solid black horizontal bars that show:
   - **Fork:** Where a single flow splits into multiple parallel activities
   - **Join:** Where multiple parallel flows merge back into a single flow
5. **External System Interaction:** Shows when MHC-PMS communicates with the *Admissions System* to find an available secure bed.
6. **End Node:** A filled black circle with a ring indicating process completion.

The Involuntary Detention Process Steps

1. **Assessment:** Doctor assesses if the patient is dangerous to self or others.
2. **Decision Confirmation:** If dangerous, the detention decision is formally confirmed.
3. **Patient Rights:** Patient must be legally informed of their rights (legal requirement).
4. **Parallel Notifications:** Simultaneously:
   - Inform the patient's next of kin
   - Notify social services/social care
   - Contact the Admissions System for bed availability
5. **Secure Accommodation:** Find and allocate a secure bed in a mental health facility.
6. **Transfer:** Physically transfer the patient to the secure facility.
7. **Documentation:** All actions are recorded in the system for legal and medical records.

Importance of This Model

1. **Legal Compliance:** Ensures that doctors and nurses follow all legally-required steps before and during detention.
2. **Audit Trail:** Provides documentation that can be used in legal proceedings.
3. **Patient Protection:** Guarantees patient rights are respected even during involuntary procedures.
4. **Process Standardization:** Ensures consistent handling across different staff members and shifts.
5. **System Requirements:** Helps developers understand what features the software must support.

Key Exam Points

- Activity diagrams show WHAT happens, not HOW the software implements it.
- Parallel bars are essential when multiple activities must occur simultaneously.
- Guards (conditions) direct the flow based on real-world decisions.

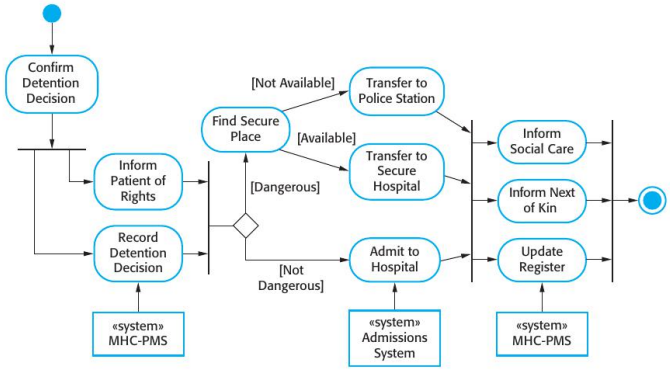


Figure 5.2 Process model of involuntary detention

Figure 5.2 is a model of an important system process that shows the processes in which the MHC-PMS is used. Sometimes, patients who are suffering from mental health problems may be a danger to others or to themselves. They may therefore have to be detained against their will in a hospital so that treatment can be administered. Such detention is subject to strict legal safeguards—for example, the decision to detain a patient must be regularly reviewed so that people are not held indefinitely without good reason. One of the functions of the MHC-PMS is to ensure that such safeguards are implemented.

Figure 5.2 is a UML activity diagram. Activity diagrams are intended to show the activities that make up a system process and the flow of control from one activity to another. The start of a process is indicated by a filled circle; the end by a filled circle inside another circle. Rectangles with round corners represent activities, that is, the specific sub-processes that must be carried out. You may include objects in activity charts. In Figure 5.2, I have shown the systems that are used to support different processes. I have indicated that these are separate systems using the UML stereotype feature.

In a UML activity diagram, arrows represent the flow of work from one activity to another. A solid bar is used to indicate activity coordination. When the flow from more than one activity leads to a solid bar then all of these activities must be complete before progress is possible. When the flow from a solid bar leads to a number of activities, these may be executed in parallel. Therefore, in Figure 5.2, the activities to inform social care and the patient's next of kin, and to update the detention register may be concurrent.

Arrows may be annotated with guards that indicate the condition when that flow is taken. In Figure 5.2, you can see guards showing the flows for patients who are

Figure 2: Process model of Involuntary Detention

Q3. With neat diagram explain data transfer use case and explain its tabular description.

Ref: Sommerville, SE 9th Ed., Chapter 5, Section 5.2.1, Figures 5.3 and 5.4

Introduction to Use Cases

Use cases are a technique for discovering and recording system requirements. They were first introduced in the Objectory method and have now become a fundamental part of UML (Unified Modeling Language). A use case describes a discrete task that involves external interaction with a system.

What is a Use Case?

A use case describes a single, complete task that a user (called an **actor**) performs with the system to achieve a specific goal. Use cases identify:

- The actors involved in the interaction
- The interaction itself (what the user does and what the system does)
- The goal or outcome of the interaction

Components of a Use Case Diagram

1. **Actor:** Represented by a stick figure — can be a human user or an external system
2. **Use Case:** Represented by an oval/ellipse containing the name of the use case
3. **System Boundary:** A rectangle that shows what's inside the system
4. **Association Lines:** Lines connecting actors to use cases they participate in
5. **Relationships:** Include «include», «extend», and generalization

Data Transfer Use Case - Overview

This use case involves a **Medical Receptionist** transferring patient data from the local MHC-PMS database to a national patient database (Patient Record System).

Purpose: To update central national health records with new or updated patient information from the local mental health clinic.

Actors Involved

- **Primary Actor - Medical Receptionist:** The human user who initiates and controls the data transfer process.
- **Secondary Actor - Patient Record System (PRS):** The external national database system that receives the transferred data.

Use Case Flow

1. Receptionist logs into the MHC-PMS system with proper credentials
2. Receptionist selects the patient whose data needs to be transferred
3. System displays patient information for verification
4. Receptionist selects "Transfer Data" command
5. System packages the patient data securely
6. System establishes connection with Patient Record System
7. Data is transmitted to the external system
8. Patient Record System validates and accepts the data
9. Confirmation message is displayed to the receptionist
10. Transfer is logged for audit purposes

Tabular Description of the Use Case

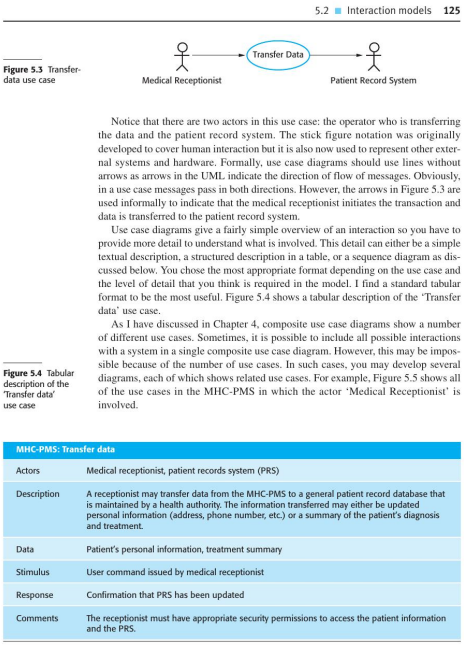
Sommerville emphasizes that a diagram alone is not sufficient — a structured tabular description provides detailed information:

Field	Details
Use Case Name	Transfer Data
Actors	Medical Receptionist (Primary), Patient Record System (Secondary)
Description	Transfer patient information and treatment summary from the local MHC-PMS to the national Patient Record System database.
Pre-conditions	• User must be logged in with appropriate permissions • Patient record must exist in local system • Network connection must be available
Post-conditions	• National database is updated with latest patient info • Transfer is logged in audit trail • Confirmation is provided to user
Data	Personal information (name, address, DOB), medical history summary, current diagnosis, treatment plan, and medication list.
Stimulus	Receptionist initiates transfer by clicking "Transfer Data" command button.
Response	Patient Record System validates data and sends "Transfer Successful" confirmation message.
Normal Flow	User selects patient → Views data → Clicks Transfer → System packages data → Sends to PRS → Receives confirmation → Displays success message
Alternative Flow	If connection fails: System displays error → Saves data locally → Queues for retry → Notifies user
Exceptions	Insufficient permissions, invalid patient data, network timeout, PRS unavailable

Comments	Requires high security clearance. All transfers must be encrypted. Audit logs are mandatory for legal compliance.

Importance of Tabular Descriptions

1. **Completeness:** Ensures all aspects of the use case are documented
2. **Clarity:** Provides unambiguous specification for developers
3. **Testability:** Test cases can be directly derived from the description
4. **Traceability:** Links requirements to implementation



5.2 ■ Interaction models 125

Figure 5.3 Transfer-data use case

Notice that there are two actors in this use case: the operator who is transferring the data and the patient record system. The stick figure notation was originally developed to cover human interaction but it is also now used to represent other external systems and hardware. Formally, use case diagrams should use lines without arrows as arrows in the UML indicate the direction of flow of messages. Obviously, in a use case messages pass in both directions. However, the arrows in Figure 5.3 are used informally to indicate that the medical receptionist initiates the transaction and data is transferred to the patient record system.

Use case diagrams give a fairly simple overview of an interaction so you have to provide more detail to understand what is involved. This detail can either be a simple textual description, a structured description in a table, or a sequence diagram as discussed below. You chose the most appropriate format depending on the use case and the level of detail that you think is required in the model. I find a standard tabular format to be the most useful. Figure 5.4 shows a tabular description of the 'Transfer data' use case.

As I have discussed in Chapter 4, composite use case diagrams show a number of different use cases. Sometimes, it is possible to include all possible interactions with a system in a single composite use case diagram. However, this may be impossible because of the number of use cases. In such cases, you may develop several diagrams, each of which shows related use cases. For example, Figure 5.5 shows all of the use cases in the MHC-PMS in which the actor 'Medical Receptionist' is involved.

Figure 5.4 Tabular description of the 'Transfer data' use case

MHC-PMS: Transfer data	
Actors	Medical receptionist, patient records system (PRS)
Description	A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Figure 3: Data Transfer Use Case

Q4. Sequence diagram for (1) View patient info (2) data transfer (3) data collection.

Ref: Sommerville, SE 9th Ed., Chapter 5 (Fig 5.6, 5.7) and Chapter 7 (Fig 7.7)

Introduction to Sequence Diagrams

Sequence diagrams are part of UML interaction diagrams that show how objects interact with each other over time. They are particularly useful for modeling the behavior of a system during specific use cases and help developers understand the dynamic aspects of a system.

Key Components of Sequence Diagrams

1. **Objects/Participants:** Shown as rectangles at the top. Can be actors, system components, or external systems.
2. **Lifeline:** A vertical dashed line extending downward from each object, representing the object's existence over time.
3. **Activation Bar:** A thin rectangle on the lifeline showing when an object is active (executing a method).
4. **Messages:** Horizontal arrows between lifelines showing communication:
   - **Synchronous messages:** Solid arrow with filled head — sender waits for response
   - **Asynchronous messages:** Solid arrow with open head — sender continues without waiting
   - **Return messages:** Dashed arrow — shows returned data or control
5. **Combined Fragments:** Boxes that show conditional logic:
   - **alt:** Alternative (if-else)
   - **opt:** Optional (if without else)
   - **loop:** Repetition

Sequence Diagram 1: View Patient Information

Purpose: Shows how a medical receptionist retrieves and views a patient's information.

Participants:

- Medical Receptionist (Actor)
- MHC-PMS User Interface
- Patient Database Controller
- Authorization System

Flow of Messages:

1. Receptionist requests to view patient info (enters patient ID)
2. UI sends viewInfo(patientID) to Controller
3. Controller requests security validation (UID verification)
4. Authorization System validates user permissions

5. If authorized: Controller queries the database for patient record
6. Database returns patient information
7. Controller formats and returns data to UI
8. UI displays patient information to receptionist

Key Points:

- Security check happens before data access
- UID (User ID) is validated for proper authorization
- Data is only returned if user has appropriate permissions

Sequence Diagram 2: Data Transfer

Purpose: Shows the interaction when transferring patient data to an external Patient Record System.

Participants:

- Medical Receptionist
- MHC-PMS System
- Patient Record System (External)

Flow of Messages:

1. Receptionist initiates transfer request
2. System retrieves patient data from local database
3. System packages data for transmission
4. System sends transferData(patientData) to Patient Record System
5. **ALT Fragment (Conditional):**
   - **[Success]:** PRS validates and stores data → Returns confirmation
   - **[Failure]:** PRS returns error message → System notifies user → Data is queued for retry
6. System logs the transfer for audit
7. Receptionist receives status notification

Key Points:

- The 'alt' box shows alternative scenarios (success vs. failure)
- Error handling is explicitly modeled
- Audit logging is mandatory for compliance

Sequence Diagram 3: Data Collection (Weather Station)

Purpose: Shows how a weather information system collects data from remote weather stations.

Participants:

- Weather Information System (requester)
- Weather Station Controller
- Individual Instruments (Thermometer, Barometer, Anemometer, etc.)

Flow of Messages:

1. Information System sends request() to Weather Station
2. Weather Station Controller receives the collection request
3. Controller enters a **LOOP** fragment to poll each instrument:
   - Sends getReading() to Thermometer → receives temperature
   - Sends getReading() to Barometer → receives pressure
   - Sends getReading() to Anemometer → receives wind speed
   - Sends getReading() to Rain Gauge → receives rainfall
4. Controller aggregates all readings
5. Controller calculates summary (average, min, max values)
6. Controller returns summarize(weatherData) to Information System

Key Points:

- Loop fragment shows repeated polling of multiple sensors
- Raw data is processed before being returned
- Summary includes computed values, not just raw readings

Advantages of Sequence Diagrams

1. **Time Ordering:** Clearly shows the order of messages
2. **Collaboration View:** Shows how objects work together
3. **Behavior Specification:** Defines expected system behavior
4. **Test Case Design:** Can be used to derive test scenarios
5. **Documentation:** Serves as reference for maintenance

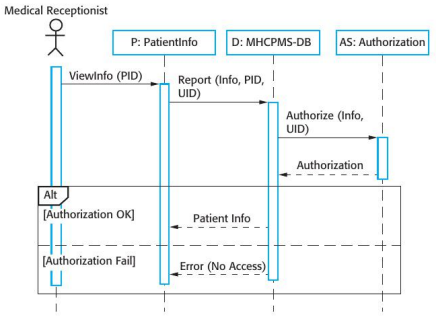


Figure 5.6 Sequence diagram for View patient information

3. The database checks with an authorization system that the user is authorized for this action.
4. If authorized, the patient information is returned and a form on the user's screen is filled in. If authorization fails, then an error message is returned.

Figure 5.7 is a second example of a sequence diagram from the same system that illustrates two additional features. These are the direct communication between the actors in the system and the creation of objects as part of a sequence of operations. In this example, an object of type Summary is created to hold the summary data that is to be uploaded to the PRS (patient record system). You can read this diagram as follows:

1. The receptionist logs on to the PRS.
2. There are two options available. These allow the direct transfer of updated patient information to the PRS and the transfer of summary health data from the MHC-PMS to the PRS.
3. In each case, the receptionist's permissions are checked using the authorization system.
4. Personal information may be transferred directly from the user interface object to the PRS. Alternatively, a summary record may be created from the database and that record is then transferred.
5. On completion of the transfer, the PRS issues a status message and the user logs off.

1. View Patient Info

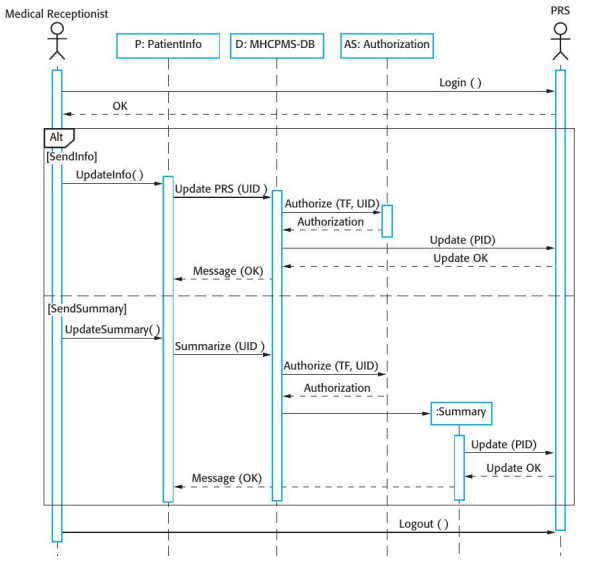


Figure 5.7 Sequence diagram for transfer data

Unless you are using sequence diagrams for code generation or detailed documentation, you don't have to include every interaction in these diagrams. If you develop system models early in the development process to support requirements engineering and high-level design, there will be many interactions which depend on implementation decisions. For example, in Figure 5.7 the decision on how to get the user's identifier to check authorization is one that can be delayed. In an implementation, this might involve interacting with a User object but this is not important at this stage and so need not be included in the sequence diagram.

2. Data Transfer

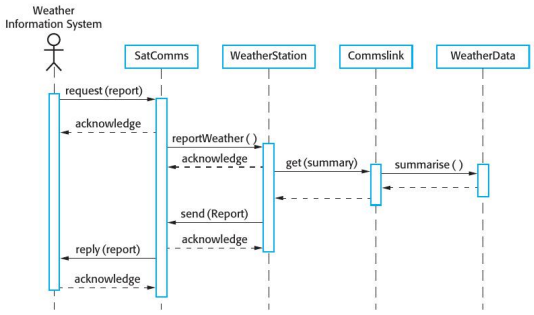


Figure 8.8 Collect weather data sequence chart

occur. If you have developed a sequence diagram to model the use case implementation, you can see the objects or components that are involved in the interaction.

To illustrate this, I use an example from the wilderness weather station system where the weather station is asked to report summarized weather data to a remote computer. The use case for this is described in Figure 7.3 (see previous chapter). Figure 8.8 (which is a copy of Figure 7.7) shows the sequence of operations in the weather station when it responds to a request to collect data for the mapping system. You can use this diagram to identify operations that will be tested and to help design the test cases to execute the tests. Therefore, issuing a request for a report will result in the execution of the following thread of methods:

SatComms:request → WeatherStation:reportWeather → Commslink:Get(summary) → WeatherData:summarize

The sequence diagram helps you design the specific test cases that you need as it shows what inputs are required and what outputs are created:

1. An input of a request for a report should have an associated acknowledgment. A report should ultimately be returned from the request. During testing, you should create summarized data that can be used to check that the report is correctly organized.
2. An input request for a report to WeatherStation results in a summarized report being generated. You can test this in isolation by creating raw data corresponding to the summary that you have prepared for the test of SatComms and checking that the WeatherStation object correctly produces this summary. This raw data is also used to test the WeatherData object.

3. Data Collection

Q5. Explain classes and association for MHC-PMS.

Ref: Sommerville, SE 9th Ed., Chapter 5, Section 5.3 and Figure 5.9

Introduction to Structural Models

According to Sommerville, structural models of software display the organization of a system in terms of the components that make up that system and their relationships. Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes.

What is a Class Diagram?

A class diagram shows the **static structure** of the system — the "things" (classes) and how they are linked (associations). It is a UML structural diagram that:

- Defines the types of objects in the system
- Shows attributes (data) and operations (methods) of each class
- Illustrates relationships between classes
- Forms the basis for database design and code structure

UML Class Notation

Each class is represented as a rectangle divided into three compartments:

1. **Top Compartment:** Class name (e.g., Patient, Consultation)
2. **Middle Compartment:** Attributes (data fields like name, date, ID)
3. **Bottom Compartment:** Operations (methods like getDetails(), prescribe())

Key Classes in MHC-PMS

According to Sommerville's Figure 5.9, the main classes are:

1. **Patient:**
   - Attributes: patientID, name, address, dateOfBirth, gender, nextOfKin
   - Represents individuals receiving mental health treatment
   - Central entity in the system
2. **Consultation:**
   - Attributes: date, time, clinic, reason, notes
   - Represents a meeting between patient and medical staff
   - Records details of each visit
3. **General Practitioner (GP):**
   - Attributes: name, practice, address, phone
   - The patient's family doctor who refers them
   - External to the mental health clinic

4. **Condition:**
   - Attributes: conditionName, severity, dateIdentified
   - The mental health problem being treated
   - Examples: Depression, Anxiety, Schizophrenia
5. **Medication:**
   - Attributes: drugName, dosage, frequency, startDate, endDate
   - Drugs prescribed during consultations
   - Linked to specific conditions

Associations and Their Meaning

Associations show relationships between classes. A line connects two classes, with labels describing the relationship.

Multiplicity in Associations

Multiplicity indicates how many objects participate in a relationship:

Symbol	Meaning	Example
1	Exactly one	Patient has 1 GP
0..1	Zero or one	Optional relationship
*	Zero or more	Patient has * Consultations
1..*	One or more	At least one required
0..*	Zero or more	Same as *

Key Associations in MHC-PMS

1. **Patient — Consultation (1 to many):**
   - A patient can have **many (1..*)** consultations over time
   - Each consultation belongs to exactly one patient
2. **Patient — GP (many to 1):**
   - Every patient is registered with **exactly one (1..1)** GP
   - One GP can have many patients
3. **Patient — Condition (1 to many):**
   - A patient may have multiple conditions
   - Each condition record belongs to one patient
4. **Consultation — Medication (1 to many):**
   - One consultation can result in **zero or more (*)** medications
   - Medications are prescribed during consultations

Importance of Class Diagrams

1. **Database Design:** Classes map to database tables
2. **Code Structure:** Classes become code classes/objects
3. **Communication:** Common language between developers and stakeholders
4. **Documentation:** Permanent record of system structure

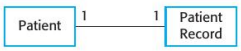


Figure 5.8 UML classes and association

by drawing a line between classes. For example, Figure 5.8 is a simple class diagram showing two classes: Patient and Patient Record with an association between them.

In Figure 5.8, I illustrate a further feature of class diagrams—the ability to show how many objects are involved in the association. In this example, each end of the association is annotated with a 1, meaning that there is a 1:1 relationship between objects of these classes. That is, each patient has exactly one record and each record maintains information about exactly one patient. As you can see from later examples, other multiplicities are possible. You can define that an exact number of objects are involved or, by using a *, as shown in Figure 5.9, that there are an indefinite number of objects involved in the association.

Figure 5.9 develops this type of class diagram to show that objects of class Patient are also involved in relationships with a number of other classes. In this example, I show that you can name associations to give the reader an indication of the type of relationship that exists. The UML also allows the role of the objects participating in the association to be specified.

At this level of detail, class diagrams look like semantic data models. Semantic data models are used in database design. They show the data entities, their associated attributes, and the relations between these entities. This approach to modeling was first proposed in the mid-1970s by Chen (1976); several variants have been developed since then (Codd, 1979; Hammer and McLeod, 1981; Hull and King, 1987), all with the same basic form.

The UML does not include a specific notation for this database modeling as it assumes an object-oriented development process and models data using objects and their relationships. However, you can use the UML to represent a semantic data model. You can think of entities in a semantic data model as simplified object classes

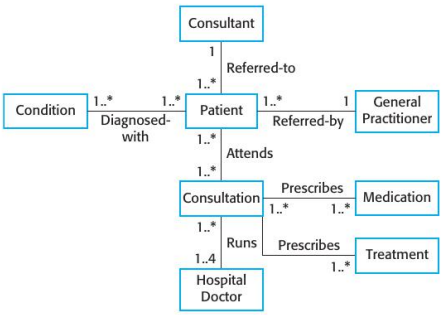


Figure 5.9 Classes and associations in the MHC-PMS

Figure 4: Classes and Associations for MHC-PMS (Sommerville Fig 5.9)

Q6. State diagram of microwave oven operation.

Ref: Sommerville, SE 9th Ed., Chapter 5, Section 5.4, Figures 5.16 and 5.17

Introduction to Behavioral Models

According to Sommerville, behavioral models are models of the dynamic behavior of a system as it executes. They show what happens when a system responds to a stimulus (event). State diagrams are used to model the behavior of a system in response to internal and external events.

What is a State Diagram?

A state diagram (also called state machine or statechart) shows:

- The different **states** a system can be in
- The **events** (stimuli) that cause state changes
- The **transitions** between states
- Actions performed during transitions

UML State Diagram Notation

- **States:** Rounded rectangles with state name
- **Initial State:** Filled black circle
- **Final State:** Filled circle with outer ring
- **Transitions:** Arrows labeled with triggering event
- **Guards:** Conditions in square brackets [condition]

States of the Microwave Oven

According to Sommerville, the microwave oven has the following states:

1. **Waiting State:**
   - Initial idle state
   - Display shows current time
   - Waiting for user input
2. **Half Power State:**
   - User has selected half power level
   - Waiting for time to be set
3. **Full Power State:**
   - User has selected full power level
   - Waiting for time to be set
4. **Set Time State:**
   - User is entering cooking duration

- Display shows entered time

5. **Operation State:**
   - Cooking is in progress
   - Timer counting down
   - Magnetron is active
6. **Disabled State:**
   - Safety state when door is opened during operation
   - All cooking stops immediately
   - Prevents radiation exposure

Stimuli (Events) and Their Effects

Event/Stimulus	Description	Effect
Half Power	User presses half power button	Transition to Half Power state
Full Power	User presses full power button	Transition to Full Power state
Number	User enters time digits	Transition to Set Time state
Start	User presses start button	Begin cooking if time set and door closed
Door Open	User opens the door	Stop cooking immediately, go to Disabled
Door Closed	User closes the door	Return to previous state
Timer Complete	Countdown reaches zero	Return to Waiting state, beep
Cancel	User presses cancel	Clear settings, return to Waiting

Key Transitions

1. **Waiting → Half/Full Power:** When power button pressed
2. **Power State → Set Time:** When number keys pressed
3. **Set Time → Operation:** When Start pressed (with door closed)
4. **Operation → Disabled:** When door opened (safety feature)
5. **Disabled → Operation:** When door closed again
6. **Operation → Waiting:** When timer completes or Cancel pressed

Safety Features Modeled

- **Door Interlock:** Cooking cannot start with door open
- **Automatic Shutoff:** Opens door immediately stops magnetron
- **State Memory:** System remembers state when interrupted

Importance of State Diagrams

1. Shows all possible system behaviors
2. Identifies missing transitions or states
3. Basis for implementing control logic
4. Used for testing — each transition should be tested

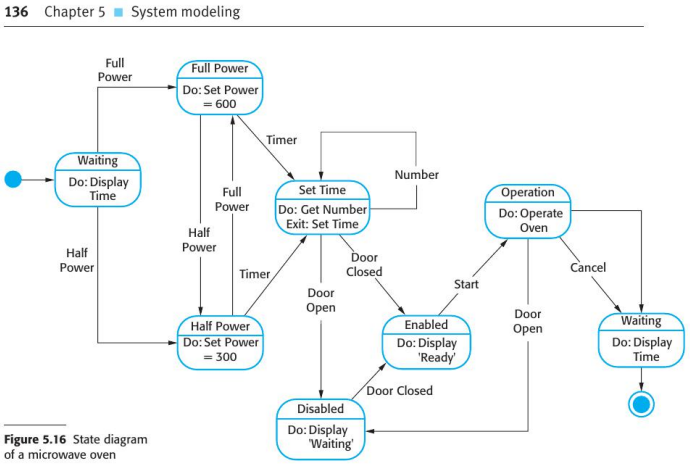


Figure 5.16 State diagram of a microwave oven

transitions from one state to another. They do not show the flow of data within the system but may include additional information on the computations carried out in each state.

I use an example of control software for a very simple microwave oven to illustrate event-driven modeling. Real microwave ovens are actually much more complex than this system but the simplified system is easier to understand. This simple microwave has a switch to select full or half power, a numeric keypad to input the cooking time, a start/stop button, and an alphanumeric display.

I have assumed that the sequence of actions in using the microwave is:

1. Select the power level (either half power or full power).
2. Input the cooking time using a numeric keypad.
3. Press Start and the food is cooked for the given time.

For safety reasons, the oven should not operate when the door is open and, on completion of cooking, a buzzer is sounded. The oven has a very simple alphanumeric display that is used to display various alerts and warning messages.

In UML state diagrams, rounded rectangles represent system states. They may include a brief description (following 'do') of the actions taken in that state. The labeled arrows represent stimuli that force a transition from one state to another. You can indicate start and end states using filled circles, as in activity diagrams.

From Figure 5.16, you can see that the system starts in a waiting state and responds initially to either the full-power or the half-power button. Users can change

Figure 5: State Machine for Microwave Oven (Sommerville Fig 5.16)

Q7. Explain use case for weather station with neat diagram.

Ref: Sommerville, SE 9th Ed., Chapter 5, Section 5.2 and Figure 5.5

Introduction to Use Cases

According to Sommerville, use case modeling was originally developed for requirements elicitation. A use case identifies the actors involved in an interaction and names the type of interaction. Use cases are documented using a high-level use case diagram and more detailed tabular descriptions.

What is a Use Case?

A use case represents:

- A specific way of using the system
- An interaction between an actor and the system
- A complete unit of functionality from the user's perspective
- The system's response to actor requests

Weather Station Context

The wilderness weather station is:

- An automated, remote, unmanned station
- Located in wilderness areas (mountains, forests)
- Collects weather data (temperature, pressure, rainfall, wind)
- Communicates via satellite link
- Must operate independently with minimal maintenance

Actors in Weather Station System

According to Sommerville, the actors are:

1. **Weather Information System:**
    - Primary actor — the main data consumer
    - Collects and processes data from multiple stations
    - Requests weather reports on a schedule
    - Stores data for forecasting and analysis
2. **Control System:**
    - Secondary actor — for maintenance and configuration
    - Used by human engineers remotely
    - Sends commands to restart, shutdown, or reconfigure
    - Monitors station health and status

Use Cases for Weather Station

According to Sommerville's Figure 5.5:

1. **Report Weather:**
   - Actor: Weather Information System
   - Description: Station sends collected weather data
   - Includes: Temperature, pressure, wind speed/direction, rainfall
   - Frequency: Typically hourly or on request
2. **Report Status:**
   - Actor: Weather Information System
   - Description: Station reports its operational status
   - Includes: Battery level, sensor health, memory usage
   - Alerts if any component is failing
3. **Restart:**
   - Actor: Control System
   - Description: Remotely restart the station software
   - Used when: Station becomes unresponsive
   - Clears memory and reinitializes sensors
4. **Shutdown:**
   - Actor: Control System
   - Description: Safely power down the station
   - Used for: Maintenance, severe weather, battery conservation
   - Ensures data is saved before shutdown
5. **Reconfigure:**
   - Actor: Control System
   - Description: Change station settings remotely
   - Examples: Reporting frequency, sensor calibration, power mode
   - Applied without physical access to station
6. **Powersave:**
   - Actor: Control System
   - Description: Switch to low-power mode
   - Reduces reporting frequency to conserve battery
   - Used during extended cloudy periods

Use Case Relationships

- **Include:** Report Weather includes Calibrate Sensors
- **Extend:** Report Status may extend to Send Alert (if critical)

Importance of Use Case Diagrams

1. **Requirements Capture:** Documents what the system must do
2. **Scope Definition:** Shows system boundaries clearly
3. **Communication:** Easy for stakeholders to understand
4. **Test Planning:** Each use case becomes a test scenario

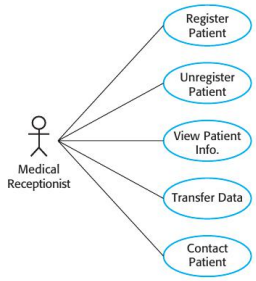


Figure 5.5 Use cases involving the role 'medical receptionist'

5.2.2 Sequence diagrams

Sequence diagrams in the UML are primarily used to model the interactions between the actors and the objects in a system and the interactions between the objects themselves. The UML has a rich syntax for sequence diagrams, which allows many different kinds of interaction to be modeled. I don't have space to cover all possibilities here so I focus on the basics of this diagram type.

As the name implies, a sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance. Figure 5.6 is an example of a sequence diagram that illustrates the basics of the notation. This diagram models the interactions involved in the View patient information use case, where a medical receptionist can see some patient information.

The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these. Interactions between objects are indicated by annotated arrows. The rectangle on the dotted lines indicates the lifeline of the object concerned (i.e., the time that object instance is involved in the computation). You read the sequence of interactions from top to bottom. The annotations on the arrows indicate the calls to the objects, their parameters, and the return values. In this example, I also show the notation used to denote alternatives. A box named alt is used with the conditions indicated in square brackets.

You can read Figure 5.6 as follows:

1. The medical receptionist triggers the ViewInfo method in an instance P of the PatientInfo object class, supplying the patient's identifier, PID. P is a user interface object, which is displayed as a form showing patient information.
2. The instance P calls the database to return the information required, supplying the receptionist's identifier to allow security checking (at this stage, we do not care where this UID comes from).

Figure 6: Weather Station Use Cases (Sommerville Fig 5.5)

Q8. Explain high level architecture of weather station.

Ref: Sommerville, SE 9th Ed., Chapter 7, Section 7.2 and Figure 7.5

Introduction to System Architecture

According to Sommerville, architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system. The weather station uses a modular subsystem architecture because it operates remotely and must be highly reliable and maintainable.

Design Considerations

The weather station architecture must address:

- **Remote Operation:** No human intervention possible
- **Reliability:** Must work 24/7 in harsh conditions
- **Power Efficiency:** Limited solar/battery power
- **Fault Tolerance:** Must continue even if parts fail
- **Maintainability:** Remote software updates possible

Architecture Style

The weather station uses a **layered/subsystem architecture** where:

- System is divided into independent subsystems
- Each subsystem has a specific responsibility
- Subsystems communicate through defined interfaces
- Failure in one subsystem doesn't crash others

The Six Major Subsystems

According to Sommerville's architecture:

1. **Data Collection Subsystem:**
   - Core function: Collects data from all sensors
   - Manages sensor polling schedule
   - Buffers data before transmission
   - Processes raw readings into standard formats
2. **Instruments Subsystem:**
   - Hardware interface layer
   - Contains drivers for each sensor type:
     - Thermometer (temperature)
     - Barometer (air pressure)
     - Anemometer (wind speed)
     - Wind vane (direction)

    - Rain gauge (precipitation)
  - Handles hardware-specific communication protocols

3. **Communications Subsystem:**
   - Manages satellite link connection
   - Handles data transmission protocols
   - Receives commands from control system
   - Manages communication failures and retries
4. **Fault Management Subsystem:**
   - Monitors all other subsystems
   - Detects failures and anomalies
   - Attempts automatic recovery
   - Reports status to control system
   - Implements watchdog timer
5. **Power Management Subsystem:**
   - Controls battery usage
   - Manages solar panel charging
   - Implements power-saving modes
   - Shuts down non-essential systems when battery low
6. **Configuration Subsystem:**
   - Stores system settings
   - Handles reconfiguration commands
   - Manages software updates
   - Stores calibration data

Key Architectural Principles

1. **Modularity:** Each subsystem is independent — if the thermometer fails, the rain gauge still works
2. **Separation of Concerns:** Each subsystem handles one aspect of functionality
3. **Graceful Degradation:** System continues with reduced functionality rather than complete failure
4. **Information Hiding:** Subsystems hide internal details from each other

Inter-Subsystem Communication

- Data Collection Instruments: Sensor readings
- Data Collection Communications: Outgoing reports
- Fault Management All: Health monitoring
- Power Management All: Power allocation

Chapter 1 ■ Exercises 25

IEEE Software, March/April 2002. This is a special issue of the magazine devoted to the development of Web-based software. This area has changed very quickly so some articles are a little dated but most are still relevant. (*IEEE Software*, **19** (2), 2002.) http://www2.computer.org/portal/web/software.

'A View of 20th and 21st Century Software Engineering'. A backward and forward look at software engineering from one of the first and most distinguished software engineers. Barry Boehm identifies timeless software engineering principles but also suggests that some commonly used practices are obsolete. (B. Boehm, *Proc. 28th Software Engineering Conf.*, Shanghai. 2006.) http://doi.ieeecomputersociety.org/10.1145/1134285.1134288.

'Software Engineering Ethics'. Special issue of IEEE Computer, with a number of papers on the topic. (*IEEE Computer*, **42** (6), June 2009.)

EXERCISES

1.1. Explain why professional software is not just the programs that are developed for a customer.

1.2. What is the most important difference between generic software product development and custom software development? What might this mean in practice for users of generic software products?

1.3. What are the four important attributes that all professional software should have? Suggest four other attributes that may sometimes be significant.

1.4. Apart from the challenges of heterogeneity, business and social change, and trust and security, identify other problems and challenges that software engineering is likely to face in the 21st century (Hint: think about the environment).

1.5. Based on your own knowledge of some of the application types discussed in section 1.1.2, explain, with examples, why different application types require specialized software engineering techniques to support their design and development.

1.6. Explain why there are fundamental ideas of software engineering that apply to all types of software systems.

1.7. Explain how the universal use of the Web has changed software systems.

1.8. Discuss whether professional engineers should be certified in the same way as doctors or lawyers.

1.9. For each of the clauses in the ACM/IEEE Code of Ethics shown in Figure 1.3, suggest an appropriate example that illustrates that clause.

1.10. To help counter terrorism, many countries are planning or have developed computer systems that track large numbers of their citizens and their actions. Clearly this has privacy implications. Discuss the ethics of working on the development of this type of system.

Figure 7: High-level Weather Station Architecture (Sommerville Fig 7.5)

Q9. Architecture for data collection.

Ref: Sommerville, SE 9th Ed., Chapter 7, Section 7.2 and Figure 7.7

Introduction

According to Sommerville, the data collection subsystem is a critical part of the weather station that manages the collection, processing, and summarization of sensor data. This subsystem must operate reliably and efficiently given the remote location and limited power resources.

Purpose of Data Collection Architecture

The data collection architecture deals with:

- How raw sensor readings are obtained
- How data is temporarily stored (buffered)
- How raw data is processed and cleaned
- How summary reports are generated
- How data is prepared for transmission

The Data Collection Process Flow

According to Sommerville, the process follows these stages:

1. **Sensor Interrogation (Polling):**
    - The software polls each sensor at regular intervals (e.g., every 5 minutes)
    - Each instrument is queried for its current reading
    - Readings include: temperature, pressure, wind speed, wind direction, rainfall
    - Timestamps are attached to each reading
2. **Data Buffering:**
    - Raw data is stored in local memory buffer
    - Buffer protects against communication failures
    - If satellite link is down, data is preserved
    - Old data is archived or overwritten when buffer is full
    - Implements circular buffer strategy
3. **Data Processing:**
    - Raw values converted to standard units (Celsius, hectopascals, km/h)
    - Calibration adjustments applied based on sensor characteristics
    - Invalid readings filtered out (spike removal)
    - Data quality checks performed
    - Outliers identified and flagged
4. **Data Summarization:**

- Statistical calculations performed over reporting period:
  - **Average:** Mean value over the period
  - **Maximum:** Highest recorded value
  - **Minimum:** Lowest recorded value
  - **Trend:** Rising, falling, or stable
- Summary packaged into report format for transmission

Main Components of Data Collection Architecture

According to Sommerville's Figure 7.7:

1. **WeatherStation (Controller):**
   - Main orchestrator of data collection
   - Implements reportWeather() and reportStatus() operations
   - Coordinates timing of collection cycles
   - Interfaces with Communications subsystem
2. **WeatherData (Data Store):**
   - Central repository for collected readings
   - Stores both raw and processed data
   - Provides access methods for retrieval
   - Manages data aging and cleanup
3. **Instrument Objects (Sensors):**
   - Individual objects for each sensor type:
     - Ground Thermometer — temperature
     - Anemometer — wind speed
     - Wind Vane — wind direction
     - Barometer — air pressure
     - Rain Gauge — precipitation
   - Each has getReading() method
   - Encapsulates hardware-specific communication

Sequence of Operations

When the Weather Information System requests data:

1. Request received by WeatherStation controller
2. Controller sends collect() message to WeatherData
3. WeatherData polls each instrument in sequence
4. Each instrument returns its current reading
5. WeatherData processes and stores readings
6. Controller calls summarize() to generate report
7. Summary returned to requesting system

Design Patterns Applied

- **Observer Pattern:** Sensors notify data store of new readings
- **Strategy Pattern:** Different processing algorithms for different data types
- **Facade Pattern:** WeatherStation provides simple interface to complex subsystem

Quality Attributes Addressed

1. **Reliability:** Buffering protects against data loss
2. **Maintainability:** Modular sensor objects easy to replace
3. **Performance:** Efficient polling minimizes power usage
4. **Accuracy:** Processing removes invalid readings

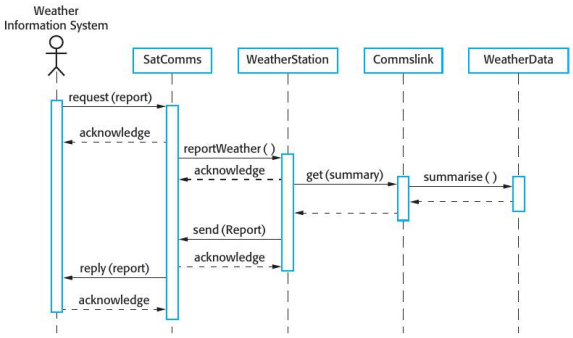


Figure 8.8 Collect weather data sequence chart

occur. If you have developed a sequence diagram to model the use case implementation, you can see the objects or components that are involved in the interaction.

To illustrate this, I use an example from the wilderness weather station system where the weather station is asked to report summarized weather data to a remote computer. The use case for this is described in Figure 7.3 (see previous chapter). Figure 8.8 (which is a copy of Figure 7.7) shows the sequence of operations in the weather station when it responds to a request to collect data for the mapping system. You can use this diagram to identify operations that will be tested and to help design the test cases to execute the tests. Therefore, issuing a request for a report will result in the execution of the following thread of methods:

SatComms:request → WeatherStation:reportWeather → Commslink:Get(summary) → WeatherData:summarize

The sequence diagram helps you design the specific test cases that you need as it shows what inputs are required and what outputs are created:

1. An input of a request for a report should have an associated acknowledgment. A report should ultimately be returned from the request. During testing, you should create summarized data that can be used to check that the report is correctly organized.
2. An input request for a report to WeatherStation results in a summarized report being generated. You can test this in isolation by creating raw data corresponding to the summary that you have prepared for the test of SatComms and checking that the WeatherStation object correctly produces this summary. This raw data is also used to test the WeatherData object.

Figure 8: Data Collection Subsystem Architecture (Sommerville Fig 7.7)

Q10. Reuse levels and costs.

Ref: Sommerville, SE 9th Ed., Chapter 16, Section 16.1 - The Reuse Landscape

Introduction to Software Reuse

According to Sommerville, software reuse is an approach to software engineering where the development process is geared to reusing existing software rather than developing new software from scratch. Software reuse has become increasingly important because of demands for lower software production costs and faster delivery of systems.

Why Reuse Software?

The benefits of software reuse include:

- **Lower development costs:** Less code to write and test
- **Faster delivery:** Reused components are already developed
- **Higher reliability:** Reused components are already tested in production
- **Reduced risk:** Known components have known behavior
- **Standards compliance:** Reused components often follow industry standards

The Reuse Landscape - Four Levels

Sommerville describes a "reuse landscape" showing the range of techniques available. From smallest to largest:

1. **Object/Function Level (Lowest Level):**
   - Reuse of individual functions, classes, or small objects
   - Examples: Math libraries, string manipulation functions, date utilities
   - Typically available in standard programming libraries
   - Lowest adaptation cost but smallest impact
   - Access through: Language libraries, utility packages
2. **Component Level:**
   - Reuse of collections of objects packaged as components
   - Examples: GUI component libraries, database access components
   - Components have well-defined interfaces
   - May require some configuration or adaptation
   - Access through: Component repositories, commercial libraries
3. **Application Level (Product Lines):**
   - Reuse of entire applications or application families
   - A base application is adapted for different customers
   - Examples: ERP systems customized for different industries
   - Requires configuration rather than programming

- Software product lines share common core with variants

4. **System Level (Highest Level):**
   - Reuse of complete systems with minimal or no modification
   - Systems are used "as-is" or with simple configuration
   - Examples: Using Microsoft Office, using a cloud service
   - COTS (Commercial Off-The-Shelf) products
   - Highest potential savings but least flexibility

Costs Associated with Reuse

Sommerville emphasizes that reuse is NOT free. Organizations must consider:

Cost Type	Description
Search Cost	Time and effort spent finding suitable reusable components. Requires browsing repositories, evaluating options, comparing alternatives.
Evaluation Cost	Assessing whether a component is fit for purpose. Includes: security review, performance testing, license checking, documentation review.
Adaptation Cost	Modifying the reused component to fit the new context. May involve: interface changes, adding features, removing unwanted functionality.
Integration Cost	Writing "glue code" to connect the reused component with other system parts. Includes: wrapper classes, adapters, data format converters.
Learning Cost	Training developers to use the reused component effectively. Understanding APIs, limitations, and best practices.
Maintenance Cost	Long-term cost of keeping the reused component updated. Dependency management, version upgrades, security patches.

When is Reuse Worthwhile?

According to Sommerville, reuse is economically justified when:

1. Total cost of (Search + Evaluation + Adaptation + Integration) $<$ Cost of developing from scratch
2. The schedule benefits of reuse outweigh any technical compromises
3. The reliability of proven components reduces testing costs
4. Maintenance burden of custom code is avoided

Challenges with Reuse

- **Not-Invented-Here Syndrome:** Developers prefer writing their own code
- **Component mismatch:** Available components don't exactly fit requirements
- **Lack of trust:** Concerns about quality of external code
- **Licensing issues:** Legal restrictions on how code can be used
- **Version dependencies:** Reused components may conflict with each other

Q11. Explain open source license models.

Ref: Sommerville, SE 9th Ed., Chapter 7, Section 7.4 - Open Source Development

Introduction to Open Source Software

According to Sommerville, open source development is an approach to software development in which the source code of a software system is published and volunteers are invited to participate in the development process. Open source software extended this idea by using the Internet to recruit a much larger population of volunteer developers.

What is Open Source?

Open source software is characterized by:

- **Source code availability:** Complete source code is freely available
- **Freedom to modify:** Anyone can change and improve the code
- **Freedom to redistribute:** Modified versions can be shared
- **Community development:** Many developers contribute improvements
- **Transparency:** Development process is visible to everyone

Why Open Source Licensing Matters

Sommerville emphasizes that although source code is "free," it is still governed by a **legally binding license** that defines:

- What you can do with the code
- What obligations you have if you use or modify it
- Whether derived works must also be open source
- How attribution must be provided

The Three Main Open Source License Models

According to Sommerville, open source licenses can be broadly categorized into three types:

1. GPL - GNU General Public License (Reciprocal/Copyleft)

- **Philosophy:** "Share and share alike" — freedom must be preserved
- **Key Rule:** If you use GPL code in your product, your **entire product** must also be released under GPL
- **Implications:**
  - All source code must be made available
  - Users must have the same freedoms you received
  - Cannot make proprietary/closed-source products using GPL code
- **Also called:** "Copyleft" license — copyright used to ensure freedom
- **Examples:** Linux kernel, GCC compiler, WordPress
- **Business Impact:** Companies often avoid GPL for commercial products

2. LGPL - GNU Lesser General Public License

- **Philosophy:** Encourage library use while protecting freedom
- **Key Rule:** You can **link to** LGPL libraries without making your whole project open source
- **Implications:**
  - Proprietary software CAN use LGPL libraries
  - Changes to the library itself must be shared
  - Users must be able to replace the library with modified versions
- **Use case:** When you want wide adoption of a library
- **Examples:** GNU C Library (glibc), Qt framework (dual licensed)
- **Business Impact:** More acceptable for commercial use than GPL

3. BSD/MIT License (Permissive)

- **Philosophy:** Maximum freedom — "Do what you want"
- **Key Rule:** You can do **almost anything** with the code, including making it proprietary
- **Requirements:**
  - Include the original copyright notice
  - Include the license text
  - Do not use the author's name for endorsement
- **What you DON'T have to do:**
  - Release your source code
  - Use the same license for your product
  - Share modifications back to the community
- **Also called:** "Permissive" license
- **Examples:** BSD operating systems, Node.js, React, Python
- **Business Impact:** Most business-friendly; widely used in commercial products

Comparison Table

Aspect	GPL	LGPL	BSD/MIT
Source disclosure	Required	Only for library changes	Not required
Proprietary use	Not allowed	Allowed (linking)	Fully allowed
Modification sharing	Required	Required for library	Not required
Commercial use	Restricted	Partially allowed	Fully allowed
License type	Copyleft	Weak copyleft	Permissive

Business and Legal Implications

According to Sommerville, organizations must be careful about open source licensing:

1. **License Contamination:** Using GPL code can "infect" your entire codebase
2. **Compliance Requirements:** Failure to comply can lead to legal action
3. **Due Diligence:** Companies need processes to track what licenses are used
4. **Strategic Choice:** License choice affects who will adopt your software

Q12. Traditional model for software testing.

Ref: Sommerville, SE 9th Ed., Chapter 8, Section 8.1 and Figure 8.3

Introduction to Software Testing

According to Sommerville, testing is intended to show that a program does what it is intended to do and to discover program defects before it is put into use. When you test software, you execute a program using artificial data. You check the results of the test run for errors, anomalies, or information about the program's non-functional attributes.

Two Distinct Goals of Testing

Sommerville distinguishes between two fundamental testing goals:

1. **Validation Testing:** To demonstrate to the developer and customer that the software meets its requirements. A successful test shows that the system operates as intended.
2. **Defect Testing:** To discover situations where the behavior of the software is incorrect, undesirable, or does not conform to its specification. A successful test reveals a defect.

The Traditional Testing Process Model

The traditional model is **staged** or **phased**. Testing progresses from small components to the complete system in a structured manner. According to Sommerville's Figure 8.3, there are three main stages:

Stage 1: Development Testing

Development testing includes all testing activities carried out by the team developing the system. It consists of three sub-stages:

1. **Unit Testing:**
   - Tests individual program components (functions, methods, classes)
   - Done by the developers who wrote the code
   - Focuses on the functionality of objects or methods
   - Uses test automation frameworks (e.g., JUnit)
   - Should test: Normal operations, boundary conditions, and error handling
2. **Component Testing:**
   - Tests composite components made of several interacting objects
   - Focuses on testing component interfaces
   - Verifies that components work together correctly
   - Tests defined interfaces and contracts
   - May be done by developers or independent testers
3. **System Testing:**
   - Tests the complete integrated system
   - Focuses on component interactions and emergent behavior
   - Verifies that the system meets functional and non-functional requirements
   - Tests: Performance, security, reliability
   - Done by a dedicated testing team (not developers)

Stage 2: Release Testing

- Performed by a **separate, independent testing team**
- System is tested as a "black box" — testers don't see source code
- Validates that the system meets requirements specified by customer
- Uses requirements-based testing approach
- Includes scenario testing with realistic use cases
- Performance testing under expected load conditions
- Goal: Ensure system is ready for delivery to customer

Stage 3: User Testing (Acceptance Testing)

- Users or customers provide input to testing
- System is tested in the user's actual environment
- Uses real data, not simulated test data
- Three types of user testing:
  - **Alpha Testing:** Users test at developer's site
  - **Beta Testing:** Users test in their own environment
  - **Acceptance Testing:** Customer decides if system is acceptable

Key Characteristics of Traditional Testing

1. **Plan-Driven:** Tests are designed based on requirements documents before coding
2. **Documentation:** Detailed test plans, test cases, and results are recorded
3. **Staged Approach:** Testing proceeds systematically from units to system
4. **Separation of Concerns:** Different teams for development and testing
5. **V-Model Alignment:** Each development phase has corresponding test phase

The V-Model Relationship

According to Sommerville, the traditional testing model follows a V-Model structure:

- Requirements → Acceptance Testing
- System Design → System Testing
- Detailed Design → Integration Testing
- Code → Unit Testing

Test plans are developed during each design phase and executed in reverse order.

Testing Principles

1. Testing can show the presence of defects, not their absence
2. Exhaustive testing is impossible — we must prioritize
3. Testing should start as early as possible
4. Defects tend to cluster in certain modules
5. Tests must be updated as the system evolves

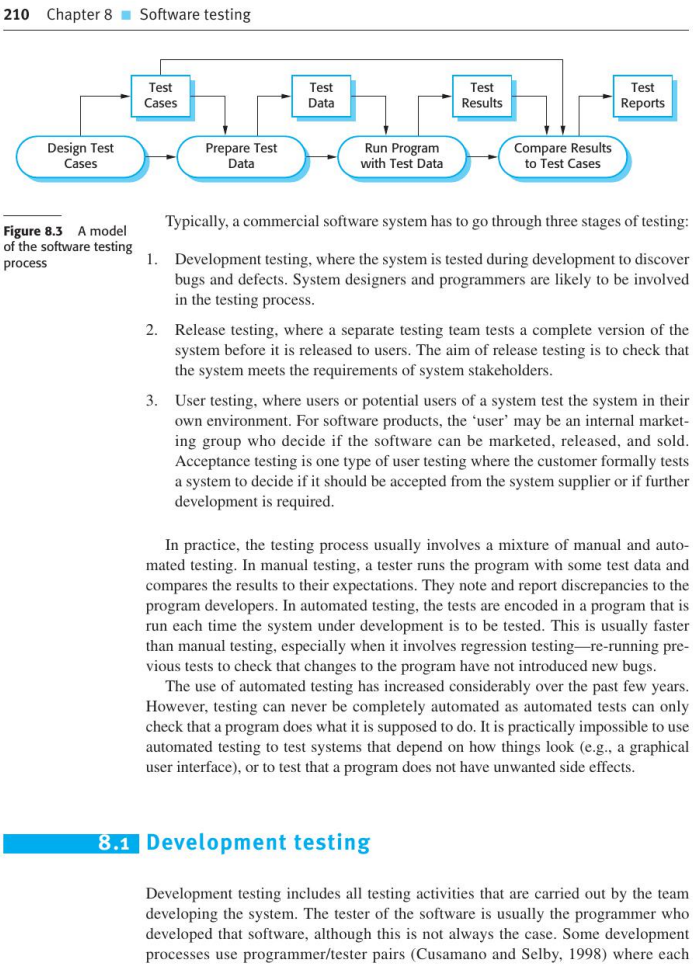


Figure 9: The Software Testing Process (Sommerville Fig 8.3)

Q13. Interface testing - types and errors.

Ref: Sommerville, SE 9th Ed., Chapter 8, Section 8.1.3

Introduction to Interface Testing

According to Sommerville, interface testing is important because interface errors are one of the most common forms of errors in complex systems. When components are combined to create larger systems or subsystems, the interfaces between components must be tested to ensure they work correctly together.

What is Interface Testing?

Interface testing focuses on:

- How different components communicate with each other
- Whether data is passed correctly between modules
- Whether components interpret shared data the same way
- Timing and synchronization between components

Why Interface Testing is Critical

- Individual components may work perfectly in isolation
- Errors only appear when components are connected
- Interface errors are often subtle and hard to detect
- They can cause system failures in unexpected situations
- Documentation may be incomplete or ambiguous

Types of Interfaces

According to Sommerville, there are different types of interfaces between components:

1. **Parameter Interfaces:**
   - Data is passed from one component to another through function/method parameters
   - Most common type of interface
   - Example: Calling a function with arguments
2. **Shared Memory Interfaces:**
   - Components share a block of memory
   - One component writes data, another reads it
   - Common in embedded systems
   - Example: Producer-consumer systems
3. **Procedural Interfaces:**
   - One component provides a set of procedures for others to call
   - Like an API (Application Programming Interface)

- Example: Library functions

4. **Message Passing Interfaces:**
   - Components communicate by sending messages
   - Common in distributed systems
   - May be synchronous or asynchronous
   - Example: Web services, microservices

Three Types of Interface Errors

According to Sommerville, interface errors fall into three main categories:

1. Interface Misuse

- A calling component calls another component incorrectly
- **Examples:**
  - Wrong number of parameters passed
  - Parameters passed in wrong order
  - Wrong data type (string instead of integer)
  - Null value passed when not allowed
- **Prevention:** Strong typing, parameter validation, good documentation

2. Interface Misunderstanding

- A calling component misunderstands the specification of the called component
- The developer makes incorrect assumptions about behavior
- **Examples:**
  - Assuming a search function returns sorted results (but it doesn't)
  - Assuming a function handles null input (but it crashes)
  - Misunderstanding units (meters vs. feet, Celsius vs. Fahrenheit)
  - Assuming thread-safety when component is not thread-safe
- **Prevention:** Clear documentation, code reviews, defensive programming

3. Timing Errors

- Occur in real-time systems with shared memory or message passing
- Components operate at different speeds
- **Examples:**
  - Producer writes data faster than consumer can read (buffer overflow)
  - Reader accesses data before writer has finished (race condition)
  - Deadlock when two components wait for each other
  - Message arrives out of order
- **Prevention:** Synchronization mechanisms, careful timing analysis, stress testing

Interface Testing Guidelines

According to Sommerville:

1. **Test with extreme values:** Pass minimum, maximum, and boundary values
2. **Test with null/empty values:** Check how interface handles missing data
3. **Test error conditions:** Force failures to check error handling
4. **Use stress testing:** High load reveals timing errors
5. **Inspect code:** Static analysis catches many interface errors
6. **Design for testability:** Build interfaces that are easy to test

Why Interface Errors are Hard to Find

- They only occur under specific conditions
- Unit tests pass but integration fails
- Errors may be intermittent (especially timing errors)
- Root cause may be in a different component than where error appears

Q14. Test driven development and benefits.

Ref: Sommerville, SE 9th Ed., Chapter 8, Section 8.2 and Figure 8.9

Introduction to Test-Driven Development

According to Sommerville, Test-Driven Development (TDD) is an approach to program development in which you interleave testing and code development. TDD was introduced as part of agile methods such as Extreme Programming (XP). However, it can also be used in plan-driven development processes.

Core Principle of TDD

The fundamental principle is: **Write the test code BEFORE you write the actual functionality**. You start with a test that defines what the code should do, then write just enough code to make that test pass.

The TDD Cycle (Red-Green-Refactor)

According to Sommerville, TDD follows an iterative cycle:

1. **Write a Test (RED Phase):**
   - Identify a small increment of functionality required
   - Write an automated test for that functionality
   - Run the test — it should **FAIL** (because the code doesn't exist yet)
   - The failing test is shown in RED by test frameworks
2. **Make the Test Pass (GREEN Phase):**
   - Write the **minimum amount of code** to make the test pass
   - Don't add extra features — only what's needed
   - Run the test — it should now **PASS**
   - Passing tests are shown in GREEN
3. **Refactor (IMPROVE Phase):**
   - Clean up the code you just wrote
   - Remove duplication, improve names, simplify logic
   - Run all tests to ensure refactoring didn't break anything
   - Repeat the cycle for the next small functionality

TDD Process in Detail

According to Sommerville's Figure 8.9, the complete TDD process is:

1. Start by identifying the increment of functionality required
2. Write a test for this functionality and implement it as an automated test
3. Run the test, along with all other implemented tests (expected to fail)
4. Implement the functionality and re-run the test
5. Once all tests run successfully, move on to implementing the next chunk

Benefits of Test-Driven Development

According to Sommerville, TDD provides several important benefits:

1. Better Code Coverage

- Every piece of code is written to make a specific test pass
- Therefore, every code segment has at least one test
- Results in comprehensive test suites
- No code exists without a corresponding test

2. Regression Testing

- A large suite of tests is built up incrementally during development
- These tests can be run every time a change is made
- Immediately detects if new code breaks existing functionality
- Provides confidence when modifying code
- Tests act as a "safety net" for refactoring

3. Simplified Debugging

- When a test fails, you know exactly which small piece of code caused it
- The problem is in the few lines you just wrote
- No need to search through large codebases
- Defects are found immediately, not weeks later
- Reduces debugging time significantly

4. System Documentation

- Tests serve as **executable documentation**
- They show how the code is intended to be used
- New developers can read tests to understand functionality
- Tests are always up-to-date (unlike written documentation)
- Provide concrete examples of expected behavior

5. Improved Design

- Writing tests first forces you to think about the interface
- Code becomes more modular and testable
- Encourages smaller, focused functions
- Leads to better separation of concerns

Challenges and Limitations of TDD

- **Learning Curve:** Requires discipline and practice
- **Time Investment:** Writing tests takes time initially (pays off later)
- **Not Suitable for All:** Difficult for UI code, legacy systems
- **Test Maintenance:** Tests need updating when requirements change

TDD vs Traditional Testing

Traditional Testing	TDD
Tests written after code	Tests written before code
Testing is a separate phase	Testing is integrated with development
May have incomplete coverage	High code coverage guaranteed
Bugs found later in process	Bugs found immediately

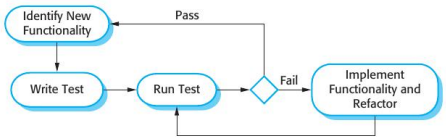


Figure 8.9 Test-driven development

The fundamental TDD process is shown in Figure 8.9. The steps in the process are as follows:

1. You start by identifying the increment of functionality that is required. This should normally be small and implementable in a few lines of code.
2. You write a test for this functionality and implement this as an automated test. This means that the test can be executed and will report whether or not it has passed or failed.
3. You then run the test, along with all other tests that have been implemented. Initially, you have not implemented the functionality so the new test will fail. This is deliberate as it shows that the test adds something to the test set.
4. You then implement the functionality and re-run the test. This may involve refactoring existing code to improve it and add new code to what's already there.
5. Once all tests run successfully, you move on to implementing the next chunk of functionality.

An automated testing environment, such as the JUnit environment that supports Java program testing (Massol and Husted, 2003), is essential for TDD. As the code is developed in very small increments, you have to be able to run every test each time that you add functionality or refactor the program. Therefore, the tests are embedded in a separate program that runs the tests and invokes the system that is being tested. Using this approach, it is possible to run hundreds of separate tests in a few seconds.

A strong argument for test-driven development is that it helps programmers clarify their ideas of what a code segment is actually supposed to do. To write a test, you need to understand what is intended, as this understanding makes it easier to write the required code. Of course, if you have incomplete knowledge or understanding, then test-driven development won't help. If you don't know enough to write the tests, you won't develop the required code. For example, if your computation involves division, you should check that you are not dividing the numbers by zero. If you forget to write a test for this, then the code to check will never be included in the program.

As well as better problem understanding, other benefits of test-driven development are:

1. *Code coverage* In principle, every code segment that you write should have at least one associated test. Therefore, you can be confident that all of the code in

Figure 10: TDD Process (Sommerville Fig 8.9)

Q15. Acceptance testing process.

Ref: Sommerville, SE 9th Ed., Chapter 8, Section 8.4 and Figure 8.11

Introduction to User Testing

According to Sommerville, user testing is a stage in the testing process in which users or customers provide input and advice on system testing. User testing is essential because influences from the user's working environment can have important effects on the reliability, performance, usability, and robustness of a system. These cannot be replicated in a developer's testing environment.

What is Acceptance Testing?

Acceptance testing is the **final stage** in the testing process before the system is accepted for operational use. It is one of three types of user testing described by Sommerville:

- **Alpha Testing:** Users work with development team to test at developer's site
- **Beta Testing:** A release made available to users for testing in their environment
- **Acceptance Testing:** Customers formally test to decide if system is ready for deployment

Purpose of Acceptance Testing

- To validate that the system meets the customer's real-world needs
- To test the system with actual user data (not simulated test data)
- To ensure the system works in the customer's operational environment
- To serve as the basis for the customer's acceptance or rejection
- To fulfill contractual obligations before final payment

The Six Stages of Acceptance Testing

According to Sommerville's Figure 8.11, acceptance testing involves six well-defined stages:

Stage 1: Define Acceptance Criteria

- Done early in the process, ideally during requirements phase
- Customer and developer jointly define what "finished" means
- Criteria should be measurable, objective, and traceable to requirements
- Examples: "Response time under 2 seconds", "99.9% uptime"
- Documented in contract or acceptance agreement

Stage 2: Plan Acceptance Testing

- Decide resources, time, and budget for acceptance testing
- Establish testing schedule and milestones
- Identify who will participate (which users, how many)
- Plan the test environment and required data
- Define coverage — which features will be tested
- Agree on process for handling discovered defects

Stage 3: Derive Acceptance Tests

- Design specific test cases based on acceptance criteria
- Tests should cover all key system functions and business-critical scenarios
- Tests linked back to requirements for traceability
- Include both positive tests (expected behavior) and negative tests (error handling)

Stage 4: Run Acceptance Tests

- Users execute the planned tests
- Testing done with **real-world data** in the actual operational environment
- Results carefully recorded and documented
- May reveal bugs not found in development testing, performance issues, usability problems
- Customer observers document any issues

Stage 5: Negotiate Test Results

- Developer and customer discuss all discovered issues
- Decide on the disposition of each defect:
  - **Must Fix:** Critical bugs fixed before acceptance
  - **Defer:** Minor issues fixed in future release
  - **Won't Fix:** Issues accepted as limitations
  - **Not a Bug:** Working as designed
- May result in a remediation plan
- Both parties must agree on resolution approach

Stage 6: Accept or Reject System

- Final decision point with three possible outcomes:
  - **Unconditional Acceptance:** System is accepted for deployment
  - **Conditional Acceptance:** Accepted with specific fixes required
  - **Rejection:** System not acceptable; significant rework needed
- Acceptance triggers final payment to developer, transfer of responsibility, start of warranty/support period

Key Differences from Other Testing

Aspect	Development Testing	Acceptance Testing
Done by	Developers/Testers	Customer/Users
Environment	Developer's lab	User's actual environment
Data	Simulated test data	Real production data
Purpose	Find and fix bugs	Validate requirements met
Outcome	Bug reports	Accept/Reject decision

Importance in Agile Methods

In agile development, acceptance testing happens incrementally. Each user story has acceptance tests defined before implementation. This ensures continuous validation throughout the project.

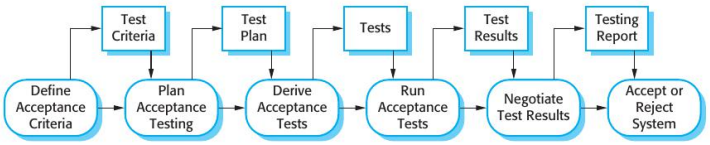


Figure 8.11 The acceptance testing process

may be a selected group of customers who are early adopters of the system. Alternatively, the software may be made publicly available for use by anyone who is interested in it. Beta testing is mostly used for software products that are used in many different environments (as opposed to custom systems which are generally used in a defined environment). It is impossible for product developers to know and replicate all the environments in which the software will be used. Beta testing is therefore essential to discover interaction problems between the software and features of the environment where it is used. Beta testing is also a form of marketing—customers learn about their system and what it can do for them.

Acceptance testing is an inherent part of custom systems development. It takes place after release testing. It involves a customer formally testing a system to decide whether or not it should be accepted from the system developer. Acceptance implies that payment should be made for the system.

There are six stages in the acceptance testing process, as shown in Figure 8.11. They are:

1. *Define acceptance criteria* This stage should, ideally, take place early in the process before the contract for the system is signed. The acceptance criteria should be part of the system contract and be agreed between the customer and the developer. In practice, however, it can be difficult to define criteria so early in the process. Detailed requirements may not be available and there may be significant requirements change during the development process.
2. *Plan acceptance testing* This involves deciding on the resources, time, and budget for acceptance testing and establishing a testing schedule. The acceptance test plan should also discuss the required coverage of the requirements and the order in which system features are tested. It should define risks to the testing process, such as system crashes and inadequate performance, and discuss how these risks can be mitigated.
3. *Derive acceptance tests* Once acceptance criteria have been established, tests have to be designed to check whether or not a system is acceptable. Acceptance tests should aim to test both the functional and non-functional characteristics (e.g., performance) of the system. They should, ideally, provide complete coverage of the system requirements. In practice, it is difficult to establish completely objective acceptance criteria. There is often scope for argument about whether or not a test shows that a criterion has definitely been met.

Figure 11: Stages of Acceptance Testing (Sommerville Fig 8.11)

Q16. Explain equivalence partitioning.

Ref: Sommerville, SE 9th Ed., Chapter 8, Section 8.1.2, Figures 8.5 and 8.6

Introduction to Partition Testing

According to Sommerville, partition testing is a systematic approach where you identify groups of inputs that have common characteristics and should be processed in the same way. The key insight is: **if two inputs produce similar behavior, testing just one of them is usually sufficient.**

What is Equivalence Partitioning?

Equivalence partitioning is a technique used to design test cases based on the idea that:

- Input data and output results can be divided into **equivalence classes** (partitions)
- All members of an equivalence class should be **treated identically** by the program
- If one member of a partition reveals a bug, all members would reveal the same bug
- If one member works correctly, all members should work correctly
- Therefore, we only need to test **one representative value** from each partition

Purpose and Benefits

- **Reduces test cases:** Instead of testing every possible input, test one from each partition
- **Systematic coverage:** Ensures all classes of inputs are tested
- **Identifies missing tests:** Helps discover partitions that might be overlooked
- **Efficient:** Maximum coverage with minimum test cases
- **Works for any data type:** Numbers, strings, dates, objects

Types of Equivalence Partitions

According to Sommerville, there are two main types of partitions:

1. Valid (Correct) Input Partitions

- These are inputs within the expected/allowed range
- The program should process them correctly
- Also called "positive" test cases
- Example: For age field (valid: 1-120), values like 25, 50, 100 are valid inputs

2. Invalid (Erroneous) Input Partitions

- These are inputs outside the expected range
- The program should handle them gracefully (error message, rejection)
- Also called "negative" test cases
- Example: For age field, values like -5, 0, 150, "abc" are invalid inputs

Steps to Apply Equivalence Partitioning

1. **Identify Input Conditions:** List all input parameters
2. **Partition Valid Inputs:** Divide valid values into groups with similar behavior
3. **Partition Invalid Inputs:** Identify types of invalid input
4. **Select Representatives:** Choose one test value from each partition
5. **Create Test Cases:** Design tests using the selected values

Example: Binary Search Function

According to Sommerville (Figure 8.6), for a `search` routine that searches a sorted array:

Input Specification:

- A sorted sequence (array) of integers
- A key value to search for
- Pre-condition: Array has at least one element

Identified Partitions:

Partition	Description	Test Case Example
1	Input with a **single value**	arr = [17], key = 17
2	Input with **even number** of values	arr = [4, 9, 21, 35], key = 9
3	Input with **odd number** of values	arr = [2, 7, 12, 20, 33], key = 12
4	Key is **first element**	arr = [1, 5, 8, 12], key = 1
5	Key is **last element**	arr = [1, 5, 8, 12], key = 12
6	Key is **middle element**	arr = [1, 5, 8, 12, 19], key = 8
7	Key is **not in array**	arr = [1, 5, 8, 12], key = 99

Boundary Value Analysis

According to Sommerville, you should also test at the **boundaries** of each partition, because:

- Many defects occur at boundary conditions
- Off-by-one errors are common programming mistakes
- Boundary values include: minimum, maximum, just below minimum, just above maximum

Boundary Testing Example: For an input field accepting values 1-100:

Boundary	Test Values	Expected Behavior
Below minimum	0	Invalid input error
At minimum	1	Valid - process normally
Typical middle	50	Valid - process normally
At maximum	100	Valid - process normally
Above maximum	101	Invalid input error

Guidelines for Equivalence Partitioning

According to Sommerville, effective partition testing requires:

1. **Complete Coverage:** Every possible input should belong to at least one partition
2. **No Overlap:** Partitions should be mutually exclusive
3. **Both Valid and Invalid:** Always test error conditions, not just happy paths
4. **Consider Combinations:** When multiple inputs exist, consider combinations
5. **Use Domain Knowledge:** Understanding the application helps identify meaningful partitions

Combined Approach for Robust Testing

For comprehensive testing, combine:

- **Equivalence Partitioning:** One value from each partition
- **Boundary Value Analysis:** Values at and near boundaries
- **Error Guessing:** Common problematic inputs (0, null, empty, special characters)

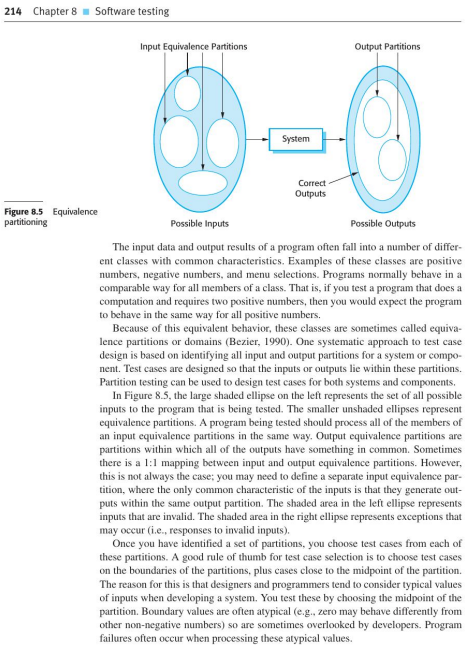


Figure 12: Input Partitions (Sommerville Fig 8.5)

All content based on: Sommerville, I. (2011). Software Engineering, 9th Edition. Addison-Wesley.